

DOCUMENTACAO DE SISTEMA

Documentação Técnica

Site institucional, painel de conteúdo (/admin) e
sistema de gestão clínica (/restrito)

Projeto	BemEstarClinic
Domínio	bemestarclinic.com
Versões analisadas	Site (APP_VERSION) 1.20.0 Gestão (SISTEMA_VERSION) 1.28.0 Pacote (package.json) 1.15.0
Situação	Em produção
Base da análise	Código-fonte em C:\Projects\SitesProjects\BemEstarClinic
Data da análise	12 de agosto de 2026

Sumário

1. Visão geral técnica	3
1.1 Objetivo do sistema	3
1.2 As três superfícies	3
1.3 Tecnologias utilizadas	3
1.4 Dependências de produção	3
2. Estrutura do projeto	5
3. Arquitetura	6
3.1 Por que um processo só	6
3.2 Ordem de tratamento das requisições	6
4. Funcionalidades técnicas	8
4.1 Publicação do site	8
4.2 Sanitização do texto vindo do painel	8
4.3 Contador de acessos com IP pseudonimizado	8
4.4 Tradução de dialeto SQL	9
4.5 Criptografia dos dados sensíveis	9
4.6 Numeração sequencial (PAC e PR)	10
4.7 Backup	10
4.8 Modo manutenção em duas camadas	10
5. APIs	11
5.1 API do site e do painel — /api/	11
5.2 Ponte entre os dois sistemas	11
5.3 API do sistema de gestão — /restrito/api/	12
6. Autenticação e segurança	14
6.1 Senhas	14
6.2 Freio de tentativas	14
6.3 Enumeração de usuário por tempo	14
6.4 IP real atrás do proxy	14
6.5 Isolamento entre as áreas	14
6.6 Cabeçalhos de resposta	14
6.7 O que nunca é servido	15
6.8 Upload	15
7. Configuração	16
8. Deploy	17

8.1 Entrega automática (GitHub Actions)	17
8.2 Deploy manual no servidor	17
8.3 nginx e systemd	17
9. Dependências e integrações externas	19
10. Tratamento de erros	19
11. Testes	20
11.1 O que não é coberto por teste automatizado	20
12. Pontos de atenção	21
12.1 Documentação interna desatualizada	21
12.2 Comentários do código desatualizados	21
12.3 Contradição interna no POSTGRES.md	21
12.4 Quatro números de versão diferentes	22
12.5 Integridade referencial ausente no banco	22
12.6 Desempenho da busca de pacientes	22
12.7 Backup sem destino externo	22
12.8 Chave de cifra é ponto único de perda	22
12.9 Histórico do repositório	23
12.10 Síntese	23

1. Visão geral técnica

1.1 Objetivo do sistema

O BemEstarClinic é um sistema único que atende três públicos distintos a partir de um só processo Node.js: o **visitante** do site institucional da clínica, o **administrador de conteúdo**, que edita o site sem tocar em HTML, e a **equipe assistencial**, que opera pacientes, agenda, anamneses e prontuário eletrônico.

A clínica atua com psicanálise, psicologia, ozonioterapia e terapias integrativas em Caruaru-PE. O sistema está em produção no domínio bemestarclinic.com.

1.2 As três superfícies

Superfície	O que é	Banco	Sessão	Indexável
/	Site público, HTML estático gerado pelo comando Publicar	— (lê arquivos em disco)	nenhuma	sim
/admin/	Painel de conteúdo do site: textos, fotos, especialidades, blog	SQLite — data/site.db	cookie sid, Path=/	noindex
/restrito/	Sistema de gestão da clínica: pacientes, agenda, anamnese, prontuário	PostgreSQL — bemestar_gestao	cookie rid, Path=/restrito	noindex

As três compartilham apenas o processo Node e a porta 5185. Bancos, sessões, layout e rotas são independentes. Uma sessão do /restrito não abre o /admin e vice-versa — com uma exceção controlada, documentada no item 5.2.

1.3 Tecnologias utilizadas

Camada	Tecnologia	Versão / origem	Observação
Runtime	Node.js	engines: >= 20; CI roda em 22	Servidor HTTP com o módulo nativo node:http, sem Express ou framework
Banco — site	SQLite	via better-sqlite3 ^12.4.1	Arquivo data/site.db
Banco — gestão	PostgreSQL	via pg ^8.22.0; CI usa postgres:16	Base bemestar_gestao, só em 127.0.0.1
Frontend público	HTML + CSS + JS	sem framework	Páginas estáticas geradas; assets/js/main.js
Frontend /admin	HTML + JS em página única	sem framework	admin/index.html (48 KB)
Frontend /restrito	HTML + JS em página única	sem framework	restrito/app.html (262 KB), tela única com módulos
Criptografia	node:crypto	nativo	scrypt para senhas; AES-256-GCM para dados
Proxy reverso	nginx	configuração versionada em nginx/	Termina TLS e encaminha para 5185
Serviço	systemd	bemestar.service	EnvironmentFile=/etc/bemestar.env
CI/CD	GitHub Actions	.github/workflows/deploy.yml	Provar -> Entregar

1.4 Dependências de produção

O projeto tem apenas **duas** dependências declaradas em package.json. Não há dependências de desenvolvimento.

Pacote	Faixa	Para que serve	Se faltar
better-sqlite3	^12.4.1	Driver do SQLite do site e do painel	db.js cai para o <code>node:sqlite</code> nativo e registra aviso; o sistema continua no ar
pg	^8.22.0	Driver do PostgreSQL do sistema de gestão	pg.js adia o erro para a primeira consulta do <code>/restrito</code> ; o site público e o <code>/admin</code> continuam funcionando

FATO

Os dois drivers são carregados dentro de `try/catch`. Essa decisão está comentada em `pg.js` como resposta a um incidente real: um `require("pg")` cru no topo derrubou o processo inteiro num servidor sem `npm ci`, tirando junto o site público, que nem usa PostgreSQL.

PONTO DE ATENÇÃO - lock desatualizado

O **package-lock.json** declara **version 1.11.0**, enquanto o `package.json` está em 1.14.4. O campo de versão do lock não afeta a instalação (o que vale são as dependências travadas), mas indica que o lock não foi regenerado nas últimas subidas de versão.

2. Estrutura do projeto

Organização da raiz do repositório. Itens marcados com (*) não são versionados.

Caminho	Tipo	Conteúdo
server.js	142 KB	Servidor HTTP, site público, painel /admin, publicação, cabeçalhos de segurança, consulta de CEP, contador de acessos
restrito.js	163 KB	Sistema de gestão inteiro: rotas, regras, perfis, modelos de anamnese, auditoria, dump SQL
pg.js	15 KB	Adaptador do PostgreSQL: tradução de dialeto, pool, transações, decifragem automática na leitura
db.js	1,5 KB	Único ponto que abre SQLite; escolhe o driver
cripto.js	7,4 KB	AES-256-GCM dos dados sensíveis e impressão digital HMAC
backup.js	11 KB	Backup diário dentro do processo (VACUUM INTO + pg_dump)
limitador.js	8,7 KB	Freio de tentativas de senha por IP e por conta
migrar.js	4 KB	Aplicador de migrations, com registro em schema_migrations
migrar-dados.js	21 KB	Migração única do SQLite para o PostgreSQL
cifrar-dados.js	6,7 KB	Cifra os registros anteriores à criptografia
index.html	67 KB	Página inicial (gerada pela publicação)
CHANGELOG.md	documento	Histórico da série do site . Criado em 12/08/2026, junto da v1.20.0
migrations/	4 arquivos	Esquema do PostgreSQL, versionado
src/	8 arquivos	Modelos das páginas internas; são a ENTRADA da publicação
admin/	1 arquivo	Painel de conteúdo
restrito/	app.html + arquivos/	Interface da gestão e anexos de paciente (*)
assets/	css, js, img, data	Estilo, scripts, imagens, índice de busca
especialidades/, blog/, profissionais/, agendar/, busca/, privacidade/	páginas geradas	Saída da publicação — não editar à mão
data/	3 arquivos (*)	site.db, limites-restrito.json e o gestao.db legado
backups/	10 arquivos (*)	Cópias automáticas
nginx/	2 arquivos	vhost do domínio e gerador de vhost
ci/	2 arquivos	Script de entrega no servidor e regra de sudo
.github/workflows/	1 arquivo	Pipeline Provar -> Entregar
deploy.sh, verificar.sh, restaurar.sh	3 scripts	Operação no servidor
testar-limitador.js, testar-campo-vazio.cjs	2 suítes	Testes automatizados
node_modules/ (*)	526 arquivos	Instalado por npm ci --omit=dev

PONTO DE ATENÇÃO - o que é gerado e o que é fonte

src/ é a fonte, as pastas de página são a saída. Editar diretamente especialidades/index.html ou agendar/index.html funciona até o próximo clique em Publicar, que sobrescreve tudo a partir de src/. O mesmo vale para o index.html da raiz, que é regravado a cada publicação.

3. Arquitetura

Um único processo Node atende as três superfícies. O que as separa não é o processo, é o roteamento: `server.js` delega para `restrito.js` tudo que começa com `/restrito`, e trata o resto por conta própria.

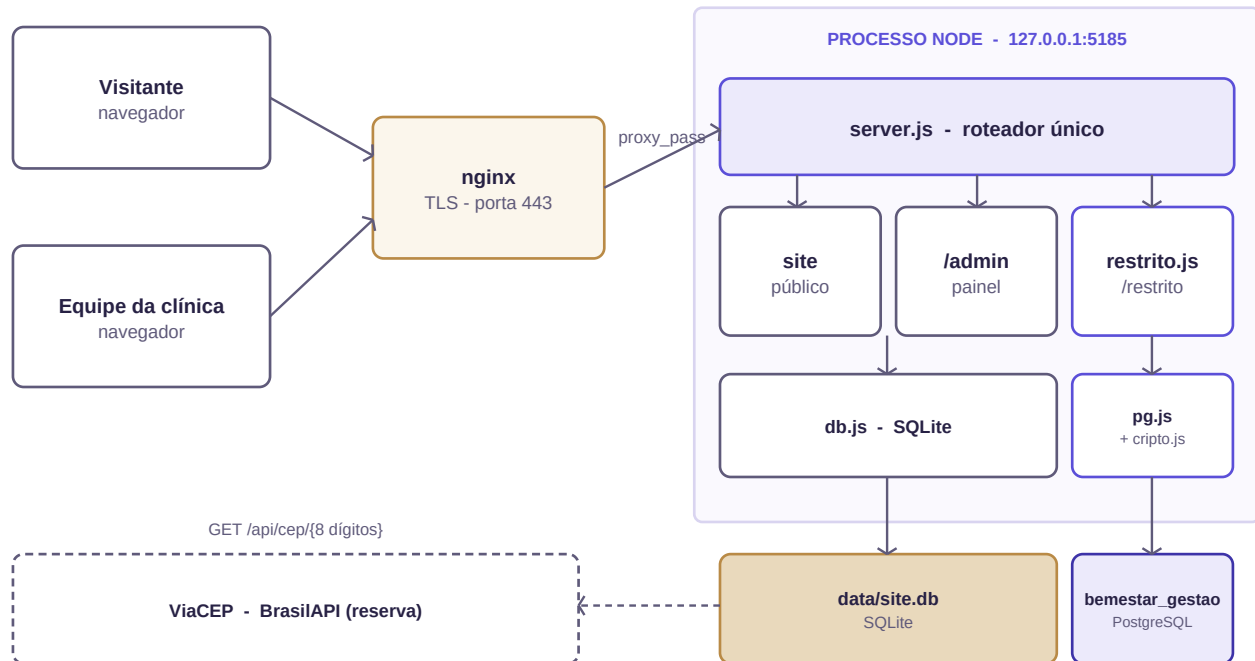


Figura 1 — Arquitetura de execução. Linha tracejada: chamada externa.

3.1 Por que um processo só

A decisão está comentada no código: o `/restrito` é um app independente que *compartilha somente a porta*. A vantagem prática é não exigir um segundo vhost nem uma segunda porta no nginx. O custo é o acoplamento de disponibilidade — e esse custo foi tratado explicitamente.

Duas defesas foram construídas para que uma falha do PostgreSQL não derrube o site:

1. **O boot não aborta.** Se a gestão não inicializa, o processo registra o erro em `ERRO_GESTAO` e sobe assim mesmo. O comentário no código diz que a versão anterior fazia `exit(1)` e isso derrubou a produção na virada para o PostgreSQL.
2. **Religação automática.** `gestaoNoAr()` tenta reconectar antes de desistir, com intervalo mínimo de 15 segundos entre tentativas. Quem chega depois que o banco voltou entra normalmente, sem intervenção.
3. Enquanto o banco não volta, **só o `/restrito`** responde 503 — com página HTML explicativa para navegador e JSON para chamadas de API, mais `Retry-After: 300`.

FATO

Essa recuperação conversa com um problema conhecido da infraestrutura: o upgrade automático do PostgreSQL derruba o banco por alguns segundos de madrugada. O aplicativo religar sozinho é o que evita que a clínica encontre o sistema fora do ar pela manhã.

3.2 Ordem de tratamento das requisições

A ordem em que `server.js` avalia cada requisição carrega regras de negócio. Ela é, na sequência:

1. cabeçalhos de segurança (todas as respostas)
2. GET /restrito/painel-do-site (ponte para o /admin)
3. 503 da gestão, se o Postgres estiver fora (só sob /restrito)
4. handleRestrito() (tudo sob /restrito)
5. MODO MANUTENÇÃO (barra o visitante)
6. /api/... (painel do site)
7. /admin/ (HTML do painel)
8. bloqueios por pasta, extensão e arquivo oculto
9. arquivo estático do disco + contagem de visita

FATO

O item 4 vem **antes** do item 5 de propósito: com o site em manutenção, a equipe continua atendendo pelo /restrito. Já o /admin (item 7) também escapa da manutenção, e quem tem sessão de administrador vê o site normal, para conferir antes de reabrir ao público.

4. Funcionalidades técnicas

4.1 Publicação do site

Nenhum texto do site é editado em HTML. O conteúdo vive no SQLite e o botão **Publicar** regenera os arquivos estáticos. O mecanismo tem três peças:

Peça	Onde	Papel
CAMPOS	server.js	Declaração única do que é editável: 19 grupos, 84 campos. Cada entrada é [chave, rótulo, tipo]. O painel monta a tela “Textos do site” lendo essa lista pela API — não há HTML de formulário escrito à mão
Marcadores	index.html e src/*.html	<!--#CHAVE--> ... <!--/CHAVE--> delimitam o trecho que cada campo substitui
migrarTextos()	server.js	No boot, popula as chaves novas lendo o HTML atual . É o que faz uma atualização nunca mudar a aparência nem exigir mexer no banco

Consequência operacional: **campo novo = 1 linha em CAMPOS + 1 marcador no HTML**. Tipos de campo suportados: input, textarea, image, lista, json_lista, stats, bigtext e endereco_cep.

PONTO DE ATENÇÃO - o que exige publicar

CSS e assets/js/main.js são servidos direto do disco e valem na hora. Já **tudo que nasce de src/ exige Publicar** — inclusive /agendar. Alterar src/agendar.html e reiniciar o serviço não muda a página no ar.

Tratamento de SEO na publicação (v1.20.0)

A publicação não apenas substitui texto: ela também **calcula** parte do que vai para a página. Três mecanismos, introduzidos na rodada de SEO de 12/08/2026:

Mecanismo	O que faz	Por quê
tituloTag()	Ajusta o <title> para a faixa de 30 a 62 caracteres. Corta primeiro na pontuação que fecha a ideia (? : — →), depois em palavra inteira; se ficar curto, acrescenta marca e cidade	Acima de ~62 o buscador corta na exibição, e o corte cai no fim — onde costuma estar a cidade. O <h1> e o og:title continuam com o título inteiro
medirImagem()	Lê a dimensão real de qualquer imagem local sob /assets/img/ e grava width e height na tag	Sem a medida, a página pula quando a imagem carrega. Antes só media a pasta de uploads, deixando de fora as fotos da galeria e dos profissionais
nivelH	O título do cartão sai como h2 nas listagens e h3 na home	O mesmo cartão é usado em dois contextos; fixo em h3 pulava de h1 para h3 nas listagens. O CSS mira as classes, então nada mudou visualmente

FATO

A extensão do medirImagem() alargou a superfície de leitura de arquivo, e o código tratou isso: o padrão aceita subpastas mas **recusa .. explicitamente**, e ainda confere que o caminho resolvido cai dentro de assets/img. É a mesma defesa em duas camadas usada na entrega de arquivos estáticos.

4.2 Sanitização do texto vindo do painel

O painel grava HTML de verdade (negrito, itálico, listas, links). O texto passa por htmlLimp(), que aceita uma lista fechada de tags (p, br, b, strong, i, em, u, s, ul, ol, li e outras) e exige que todo href case com ^((https?://|mailto:|tel:|/|#)). Os campos em ESPECIAIS (listas estruturadas) e o endereço escapam do filtro porque têm formato próprio.

4.3 Contador de acessos com IP pseudonimizado

O site conta visitas sem guardar IP em claro. Grava-se sha256(VISIT_SALT + ip), com o sal gerado por instalação e persistido em settings. Sem o sal, o hash de um IPv4 seria quebrável por força bruta — só existem 4

bilhões de endereços.

- Uma visita por IP a cada **30 minutos**; depois disso conta como visita nova.
- Robôs são descartados por uma lista de agentes (inclui GPTBot, ClaudeBot, PetalBot).
- Só páginas HTML contam — CSS, imagem, sitemap e robots não.
- **Retenção de 12 meses**, com limpeza automática, para atender à exigência da LGPD de prazo definido.

PONTO DE ATENÇÃO - pendência herdada

O `visit_salt` desta instalação **esteve exposto em repositório público** junto com o `data/site.db` no histórico de commits. Enquanto o sal não for rotacionado, a promessa da política de privacidade de que o IP é irrecoverável não se sustenta para os registros já gravados.

4.4 Tradução de dialeto SQL

A migração para PostgreSQL não reescreveu as 124 consultas do sistema de gestão. O adaptador `pg.js` traduz o SQL em tempo de execução, percorrendo o texto uma vez e respeitando strings, identificadores entre aspas e comentários:

Tradução	Motivo
? vira \$1, \$2...	Marcador de parâmetro do SQLite para o do PostgreSQL
LIKE vira ILIKE	No SQLite o LIKE ignora maiúsculas para ASCII; no PostgreSQL, não. Sem essa tradução, procurar "maria" pararia de achar "Maria" — e a busca de paciente morreria sem erro nenhum

A API do adaptador foi deliberadamente **renomeada** (`Q.get` / `Q.all` / `Q.run` / `Q.inserir` / `Q.exec` / `Q.tx` / `Q.bruto`) em vez de imitar a do `better-sqlite3`. O raciocínio está comentado: o driver antigo é síncrono e o novo é assíncrono; se a API fosse igual, um `await` esquecido devolveria uma Promise, `if` (linha) passaria e o sistema gravaria errado **em silêncio**. Com o nome diferente, o que ficou por converter quebra na hora.

FATO

Dois analisadores de tipo são registrados no driver: **bigint (OID 20)** e **numeric (OID 1700)** passam a voltar como número. Sem isso, `COUNT(*)` voltaria como a string "0" — que é *verdadeira* em JavaScript. Todo `if (conta(...))` do sistema passaria a disparar com zero vínculos: o sistema recusaria excluir cadastro sem vínculo e nunca criaria o login inicial, por achar que já existe admin.

4.5 Criptografia dos dados sensíveis

Campos de dado pessoal sensível são cifrados **pela aplicação**, antes de chegarem ao banco. A chave vive no ambiente do serviço, nunca no banco.

Aspecto	Implementação
Algoritmo	AES-256-GCM — cifra autenticada: adulteração no banco vira erro, não lixo silencioso
IV	12 bytes aleatórios por valor. Dois pacientes com o mesmo telefone geram textos cifrados diferentes
Formato	<code>enc:1:<iv>:<tag>:<cifrado></code> , tudo em base64. O prefixo permite reconhecer o que já está cifrado; o "1" é a versão da chave
Chave	DADOS_CHAVE — 32 bytes em base64. DADOS_CHAVE_ANTERIOR permite rotação sem parar o sistema
Decifragem	Centralizada em <code>pg.js</code> , no único caminho por onde toda leitura passa. Funciona com JOIN, apelido de coluna e subconsulta, porque o valor se identifica pelo prefixo
Falha	Chave errada ou valor adulterado devolvem <code>[protegido]</code> em vez de estourar — a tela abre e o resto do cadastro continua utilizável
Busca exata	HMAC-SHA256 do valor normalizado ("impressão digital"). HMAC e não hash puro: um SHA256 de CPF é reversível na prática, são só 11 dígitos

O que **NÃO** é cifrado, por decisão explícita: nome e código do paciente, porque são as chaves de busca e ordenação das listas; datas usadas em filtro de período; status; ids; e os liga-desliga. O comentário no código argumenta que um nome isolado, sem CPF, endereço, telefone e prontuário, é o que já aparece na agenda impressa sobre o balcão.

PONTO DE ATENÇÃO - consequência de desempenho

A busca por CPF **deixou de acontecer no SQL** e passou para a aplicação, depois de decifrar — cada CPF tem texto diferente no banco, não há o que comparar. Para pacientes, a lista inteira é trazida do banco e filtrada em memória. Isso preserva a busca por parte do número, mas **a lista de pacientes não pagina no banco**: quem pagina é a tela. O custo cresce com o cadastro.

4.6 Numeração sequencial (PAC e PR)

Pacientes recebem PAC-AAAA-00000 e prontuários recebem PR-AAAA-00000. Os dois compartilham o mesmo motor (`proximoSequencial / emitirSequencial`), com três camadas:

1. **Contador persistente** em `g_config` (`pront_seq_<ano>`, `pac_seq_<ano>`), que só sobe. Isso impede o reaproveitamento do número de um registro excluído — defeito que existiu quando a emissão usava `MAX()`.
2. A emissão usa `max(contador, MAX no banco) + 1`, o que também recupera a sequência se o `g_config` sumir.
3. **Índice único parcial** no banco como última defesa: barra duplicata até em INSERT direto, ou num backup restaurado por cima.

Nenhuma das duas colunas está na whitelist TAB, então o CRUD genérico jamais as grava: número enviado pelo cliente é ignorado.

4.7 Backup

Banco	Método	Arquivo	Por que assim
site.db	VACUUM INTO	site.AAAAMMDD-HHMMSS.db	Backup online do SQLite. Copiar o .db com <code>cp</code> não serve: o WAL fica em outro arquivo
bemestar_gestao	pg_dump	bemestar_gestao.AAAAMMDD-HHMMSS.sql	Dump restaurável; sai com <code>--clean --if-exists</code>

- Roda **dentro do processo**, sem cron: de hora em hora pergunta se já passaram 24h. Se a máquina estava desligada na hora marcada, a cópia sai no próximo boot em vez de ser pulada.
- A cópia do SQLite é **aberta e submetida a integrity_check** antes de contar como válida; cópia quebrada é apagada e vira erro no log.
- Guarda as 30 últimas de cada banco em backups/. Ajustável por `BACKUP_HORAS` e `BACKUP_MANTER`.
- O **gestao.db legado saiu da lista** de bancos copiados, com o comentário de que copiá-lo daria a impressão de haver backup atualizado de prontuários quando o conteúdo pararia na data da virada.

4.8 Modo manutenção em duas camadas

Camada	Quando age	Resposta
Aplicação	Botão ligado no painel, app rodando	HTTP 503 + <code>Retry-After 3600</code> para o visitante. Admin logado vê o site normal; <code>/admin</code> , <code>/api</code> , <code>/assets</code> e <code>/well-known</code> passam
nginx	Aplicação fora do ar (deploy, restart, queda)	<code>error_page 502 503 504 serve manutencao.html</code> do disco

O arquivo `manutencao.html` é regravado sempre que a mensagem muda no painel, e também no primeiro boot — o nginx precisa dele em disco justamente quando não há aplicação para gerá-lo.

5. APIs

Duas APIs REST-like independentes, ambas em JSON. Nenhuma delas usa framework de roteamento: as rotas são comparações e expressões regulares sobre o caminho.

5.1 API do site e do painel — /api/

Autenticação por cookie sid. Tudo abaixo de /api/ exige sessão, com duas exceções marcadas.

Método	Rota	Finalidade	Corpo / parâmetros	Resposta
POST	/api/login	Público. Autentica no painel	{ password }	200 + cookie sid; 401 senha incorreta; 429 no freio
GET	/api/cep/{8 dígitos}	Público. Consulta CEP	8 dígitos no caminho	200 dados; 400 formato; 404 não achado; 429 limite; 503 fonte fora
GET	/api/me	Confirma sessão e informa a versão	—	{ ok, version }
GET	/api/stats	Estatísticas de acesso do site	—	Agregados de visitas
GET / POST	/api/manutencao	Lê e altera o modo manutenção	{ ligar, titulo, texto }	{ ok, ligado, titulo, texto }
POST	/api/logout	Encerra a sessão no servidor	—	200
POST	/api/password	Troca a senha do painel	{ current, next }	200; 400 senha atual errada, menos de 8 caracteres ou igual à atual; 429 no freio
GET	/api/content	Todo o conteúdo + a definição de CAMPOS	—	settings, campos, services, portfolio, testimonials, team, posts
PUT	/api/settings	Grava textos do site	objeto chave/valor (só chaves de CAMPOS)	200
POST	/api/{tabela}	Cria registro	campos da tabela	200
PUT	/api/{tabela}/:id	Altera registro	campos da tabela	200
DELETE	/api/{tabela}/:id	Exclui registro	—	200
POST	/api/upload	Envia imagem	{ name, dataUrl }	200 + caminho; 400 se não for PNG/JPG/WEBP/GIF
POST	/api/publish	Regenera o site estático	—	200 + resumo da publicação

{tabela} aceita apenas: services, portfolio, testimonials, team, posts. As colunas graváveis vêm da whitelist TABLES; o que não estiver nela é descartado. Para services e posts, o slug é derivado do título e recebe sufixo se colidir.

5.2 Ponte entre os dois sistemas

Método	Rota	Finalidade	Regras
GET	/restrito/painel-do-site	Abre o /admin já autenticado, a partir do sistema de gestão	Exige sessão do /restrito com perfil admin (403 para secretaria e profissional); recusa Sec-Fetch-Site: cross-site; cria a sessão sid no servidor, sem token na URL; registra na auditoria

FATO

A rota mora sob **/restrito**, e não sob **/admin**, porque o cookie da gestão é gravado com Path=/restrito — o navegador não o enviaria para /admin. O comentário registra que a primeira versão errou nisso e que o teste automatizado não pegou, por montar o cabeçalho de cookie à mão.

5.3 API do sistema de gestão — /restrito/api/

Autenticação por cookie rid (sessão de 8h). Toda rota abaixo exige sessão, exceto o login. As permissões por perfil estão no capítulo 6.

Sessão, conta e informações do sistema

Método	Rota	Finalidade
POST	login	Autentica. Trava por IP e por conta; tempo constante mesmo para usuário inexistente
GET	me	Perfil, nome, e o profissional_id / profissional_nome quando o usuário é profissional
POST	logout	Encerra a sessão no servidor e também a do painel do site neste navegador
POST	senha	Troca a própria senha; derruba as demais sessões
GET	painel	Números da tela inicial
GET	sobre	Versão, histórico de versões, tecnologias e banco ativo (admin)
GET	backup/sql	Dump completo por pg_dump, transmitido direto ao navegador (admin)
GET	auditoria	Trilha de auditoria, com filtros de período, pessoa, ação e módulo (admin)
GET	auditoria/:id	Detalhe de um evento, campo a campo (admin)
GET	modelos	Os três modelos de anamnese (fonte única de tela e impressão)
POST	upload	Anexo de paciente; gravado em restrito/arquivos

CRUD genérico

Uma única rota, {tabela}/{:id}, atende dez tabelas. A tabela precisa existir na whitelist TAB e as colunas graváveis são as declaradas lá — é essa lista que impede o cliente de gravar campos que são do servidor.

Tabela	Colunas fora da whitelist (só o servidor grava)
pacientes	codigo, arquivado, inativo_em, inativo_motivo
atendimentos	prontuario_id — quem liga o agendamento à pasta é o servidor
prontuario	numero, status, alta_em, alta_motivo, arquivado
anamneses	status, finalizada_em, prontuario_id
convenios, procedimentos, salas, profissionais, prontuario_registros, documentos_gestao	sem restrição além das colunas listadas em TAB

Parâmetros de consulta aceitos no GET de lista:

Parâmetro	Efeito
?q=	Busca. Em pacientes casa nome, código e CPF (com ou sem máscara, mínimo de 3 dígitos); em prontuario casa número, especialidade, profissional e os pacientes que casarem; nas demais, o campo nome
?paciente_id=	Filtra pelo paciente
?prontuario_id=	Filtra os lançamentos de uma pasta
?arquivados=	ausente = só o que não está arquivado; 1 = inclui; so = apenas os arquivados
?status=	Ativo ou Alta, nas tabelas que têm status
?ativo=1 0	Relação de pacientes ativos ou inativos
?de= & ?ate=	Período, em relatórios (validado por ^\d{4}-\d{2}-\d{2}\$ antes de entrar na consulta)

Rotas de regra de negócio

Método	Rota	Finalidade
POST	anamneses/:id/finalizar	Fecha a anamnese e ABRE o prontuário (ou reaproveita a pasta existente do par paciente + procedimento) e recolhe os agendamentos soltos daquele par
POST	anamneses/:id/reabrir	Volta a Rascunho. Não desfaz o prontuário, que já pode ter lançamentos
POST	prontuario/:id/alta	Encerra o acompanhamento naquela especialidade
POST	prontuario/:id/reabrir	Reativa a pasta
POST / DELETE	prontuario/:id/atendimentos/:aid	Liga ou solta um agendamento da pasta
GET	prontuario/:id/vinculos	Anamneses e agendamentos ligados à pasta
GET	prontuario/:id/disponiveis	Agendamentos que poderiam ser vinculados
GET	pacientes/:id/prontuarios	As pastas do paciente (alimenta os chips de número)
POST	pacientes/:id/inativar	A pessoa deixou a clínica. Avisa quantas pastas ficaram em tratamento, mas não impede
POST	pacientes/:id/reactivar	Volta a ativa
POST	{pacientes prontuario prontuario_registros}/:id/arquivar	Tira da lista sem apagar nada
POST	{...}/:id/restaurar	Devolve à lista de origem
GET	historico/:patientId	Prontuário completo: anamneses, pastas, lançamentos e atendimentos em ordem cronológica. Alimenta a impressão do histórico
GET	historico/{paciente prontuario}/:id	Linha do tempo de eventos da entidade
GET	relatorios	Movimento da clínica (aceita ?de=&ate=)
GET	relatorios/pacientes	Relação de ativos/inativos com endereço, WhatsApp, quem assiste e especialidades
GET/POST/PUT/DELETE	usuarios[:id]	Contas do sistema (admin)

FATO

Um vínculo **não** é aceito pelo CRUD: `atendimentos.prontuario_id` é preenchido por `sincronizarProntuarioDoAtendimento()`, que só refaz o vínculo se o procedimento mudou naquela edição — comparando o estado lido *antes* do UPDATE. Sem essa comparação, um vínculo feito à mão seria desfeito sem querer a cada edição do agendamento.

6. Autenticação e segurança

Nenhum segredo, senha, chave ou token real aparece neste documento. O que segue é o mecanismo.

6.1 Senhas

Aspecto	Implementação (idêntica nos dois sistemas)
Derivação	scrypt com N=16384, r=8, p=1, keylen=32 — 16 MB de memória por tentativa, o que inviabiliza ataque em escala com GPU
Sal	16 bytes aleatórios por senha
Formato	scrypt\$N\$r\$p\$sal\$derivado, em hexadecimal
Comparação	crypto.timingSafeEqual — comparar com === vaza informação pelo tempo
Migração	Hash antigo em sha256 puro ainda é aceito e sobe para scrypt no primeiro login correto , sem exigir troca de senha

6.2 Freio de tentativas

limitador.js mantém **dois baldes**: um por IP e outro por conta. O balde por conta soma os erros de todos os endereços — é o que detecta o ataque distribuído, que passava despercebido pela trava por IP. O estado é persistido em arquivo JSON, então reiniciar o serviço não zera o freio.

Protege seis pontos de entrada, entre eles o login do painel, o login da gestão e as duas trocas de senha — porque a rota de troca **recebe a senha atual**: sem freio, quem roubasse um cookie poderia adivinhá-la por ali, contornando o login.

6.3 Enumeração de usuário por tempo

No login da gestão existe HASH_ISCA: um hash scrypt descartável, gerado no boot, que é verificado quando o login **não existe**. Sem isso, usuário inexistente respondia em cerca de 1 ms e usuário existente em cerca de 100 ms — diferença suficiente para descobrir quem tem conta.

6.4 IP real atrás do proxy

O cabeçalho X-Forwarded-For é texto que o cliente também escreve; o nginx **acrescenta** ao fim da lista em vez de substituir. Ler o primeiro item, como o código fazia, era ler exatamente o que o visitante digitou — e isso anulava a trava de força bruta.

A correção, presente nos dois arquivos: o cabeçalho só é aceito quando a conexão veio de fato do nginx local (127.0.0.1 ou ::1), e usa-se o X-Real-IP, que o nginx **sobrescreve**, ou, na falta dele, o **último** item da lista.

6.5 Isolamento entre as áreas

- Cookies **HttpOnly**, **SameSite=Lax**, com **Path isolado** (/ para o painel, /restrito para a gestão) e **Secure** quando a requisição chegou por HTTPS.
- Sessão do painel expira em **12h**; a da gestão, em **8h**.
- Trocar a senha **derruba as demais sessões** — um cookie roubado para de valer no momento da troca.
- Bloquear um profissional **derruba a sessão aberta dele na hora**.
- A única travessia possível entre os dois sistemas é a ponte do item 5.2, restrita ao perfil admin e registrada na auditoria.

6.6 Cabeçalhos de resposta

Cabeçalho	Site público	/admin	/restrito
X-Content-Type-Options: nosniff	sim	sim	sim
X-Frame-Options: SAMEORIGIN	sim	sim	sim

Cabeçalho	Site público	/admin	/restrito
Referrer-Policy: strict-origin-when-cross-origin	sim	sim	sim
Permissions-Policy (câmera, microfone, geolocalização)	sim	sim	sim
Strict-Transport-Security (1 ano, includeSubDomains)	sim*	sim*	sim*
Content-Security-Policy	não	CSP_PAINEL	CSP_GESTAO
X-Robots-Tag: noindex, nofollow	não	sim	sim
Cache-Control: no-store	não	sim	sim

* Emitido apenas quando `x-forwarded-proto = https`. Emitir HSTS em HTTP puro travaria o acesso em ambiente sem certificado.

INFERÊNCIA

O site público não tem CSP. Isso é uma decisão consciente registrada no projeto, não um esquecimento: a página usa estilo inline, imagens do Unsplash, Google Fonts, Maps e Doctoralia, e uma política mal calibrada quebraria o site. O risco é menor porque não há sessão nem dado sensível no site público. Fechar exige inventariar as origens e testar página a página.

6.7 O que nunca é servido

Regra	Alcance
Pastas bloqueadas	/data, /src, /node_modules, /nginx, /backups, /restrito/arquivos
Extensões bloqueadas	js, json, md, db, log, bak, sqlite, yaml, toml, lock, sh , bash, service , env , conf, ini, sql, pem, key, crt, backup, old, orig, swp, tmp — exceto sob /assets/
Arquivos ocultos	Qualquer caminho com componente iniciado por ponto, exceto /.well-known/ (necessário para o Let's Encrypt)
Travessia de caminho	<code>path.normalize</code> + verificação de <code>startsWith(ROOT)</code> ; nos anexos, <code>path.basename</code>
Anexos de paciente	Servidos apenas com sessão válida, com Cache-Control: <code>private</code> , <code>no-store</code> e <code>noindex</code>
Camada do nginx	<code>location ~ /\.(git env gitignore)</code> fecha antes mesmo de chegar na aplicação

FATO

A lista de extensões cobre `.sh` porque um `deploy.sh` servido em HTTP 200 entrega o nome do serviço, o usuário do `systemd`, o caminho da aplicação e onde ficam os backups. Esse foi um achado real de auditoria neste projeto.

6.8 Upload

Os dois sistemas aceitam apenas **PNG, JPG, WEBP e GIF**, validados pelo prefixo do data URL. **SVG é recusado de propósito**: é XML, aceita `<script>` dentro e seria servido da mesma origem do site — um arquivo desses vira XSS armazenado com acesso ao cookie de quem abrisse o link. O nome do arquivo é reescrito (slug + carimbo de tempo), nunca aproveitado como veio.

7. Configuração

Nenhuma credencial está no repositório. Em produção as variáveis vêm do systemd, por `EnvironmentFile=/etc/bemestar.env`. Em desenvolvimento, um arquivo `.env` na raiz serve; ele não entra no git.

Variável	Obrigatória	Padrão	Para que serve
DADOS_CHAVE	sim	—	32 bytes em base64. Cifra CPF, RG, endereço, telefone, e-mail, anamnese, prontuário e auditoria. O serviço se recusa a subir sem ela
DADOS_CHAVE_ANTERIOR	não	—	Só durante uma rotação: mantém a leitura do que foi escrito com a chave antiga
PGHOST	não	127.0.0.1	Servidor PostgreSQL
PGPORT	não	5432	Porta
PGUSER	não	bemestar	Usuário
PGPASSWORD	sim	vazio	Senha do banco
PGDATABASE	não	bemestar_gestao	Base
DATABASE_URL	não	—	Alternativa única; tem prioridade sobre as cinco acima
PGPOOL_MAX	não	10	Conexões no pool
PORT	não	5185	Permite subir uma cópia de teste sem derrubar a do cliente
BACKUP_HORAS	não	24	Intervalo entre cópias
BACKUP_MANTER	não	30	Cópias guardadas por banco
PG_DUMP	não	pg_dump	Caminho do binário
BEMESTAR_ENV	não	—	Caminho explícito do arquivo de ambiente
PORTA_TESTE	não	5197	Porta usada pela suíte de testes

Ordem de carregamento (`carregarAmbiente`): o que já está no ambiente sempre vence — é o caso do serviço; depois `/etc/bemestar.env`; por último o `.env` local. Isso existe porque `deploy.sh`, `restaurar.sh` e `verificar.sh` rodam no shell do usuário, fora do serviço, onde as variáveis do systemd não existem.

PONTO DE ATENÇÃO - permissão do arquivo de ambiente

Divergência na documentação existente. O cabeçalho de `.env.exemplo` diz que `/etc/bemestar.env` é `root:root`, `chmod 600`; o `POSTGRES.md` instrui `chown root:deploy` e `chmod 640`, com a justificativa de que o serviço roda como `deploy` e que `600` impediria rodar `verificar.sh` e os scripts de migração sem `sudo`. As duas instruções não podem estar certas ao mesmo tempo. O `POSTGRES.md` é o mais recente e o que traz o raciocínio.

NÃO IDENTIFICADO

O arquivo `DEPLOY.md`, citado pelo `.env.exemplo`, **não existe no repositório**. O conteúdo equivalente está em `POSTGRES.md` e no `README.md`.

8. Deploy

8.1 Entrega automática (GitHub Actions)

O pipeline tem dois trabalhos em sequência. O segundo só existe se o primeiro passar — uma suíte vermelha não vira site no ar.

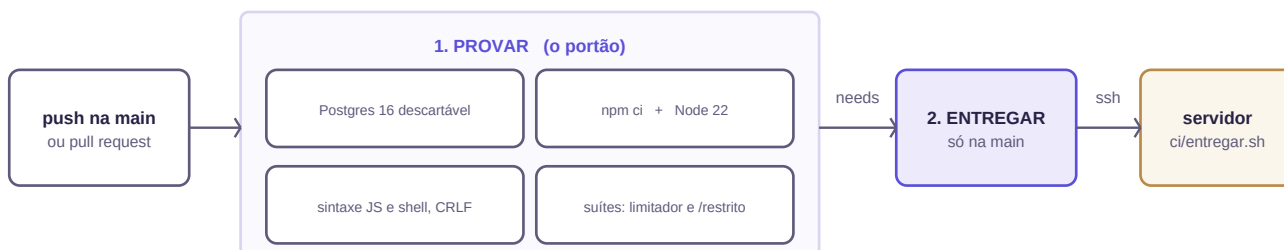


Figura 2 — Pipeline de entrega. O trabalho 2 só existe se o 1 passar.

FATO

A chave de entrega não dá shell. No servidor ela está presa a um `command=` que roda apenas a entrega deste site. Isso foi provado antes de ser ligado: pedir `cat /etc/shadow`, pedir terminal e abrir túnel para a porta do PostgreSQL foram todos recusados.

Conferências estáticas do portão: todo arquivo `.js` e `.cjs` passa por `node --check`; todo `.sh` passa por `bash -n`; e nenhum `.sh` pode ter fim de linha do Windows — um `\r` faz o `bash` ler “`bash\r`” como interpretador e o script morre apontando para um arquivo que existe.

A chave de cifra usada na prova é gerada na hora com `openssl rand -base64 32` e morre com a execução. O comentário registra que uma frase qualquer no lugar dela fez toda rota do `/restrito` responder 503 — sintoma que *parece* falta de Postgres.

8.2 Deploy manual no servidor

```

./verificar.sh      # só LÊ: commit, driver, permissões, contagens, backups
sudo ./deploy.sh    # backup -> para -> protege bancos -> pull -> npm ci
#                  -> devolve -> sobe -> confere
  
```

Regra de operação	Motivo
Nunca <code>git pull</code> puro	Os bancos não são versionados. Um commit antigo que os removeu pode apagá-los. O <code>deploy.sh</code> tira <code>site.db</code> , <code>gestao.db</code> e <code>restrito/arquivos</code> do caminho antes do <code>pull</code>
<code>git pull</code> não reinicia o Node	Alterou <code>server.js</code> ou <code>restrito.js</code> ? Reinicie. Senão o processo segue com o código velho em memória — e o sintoma típico é uma função nova responder “Rota não encontrada”
<code>git pull</code> não instala dependência	O <code>deploy.sh</code> roda <code>npm ci --omit=dev</code> no passo 6
Editou texto ou foto no painel?	Precisa clicar em Publicar
<code>verificar.sh</code> compila o script do <code>app.html</code>	O JavaScript da gestão mora dentro de um template literal. Uma crase mal colocada num comentário já fechou a string e deixou a tela em branco sem erro no console nem no log — duas vezes

8.3 nginx e systemd

- A aplicação escuta **apenas em 127.0.0.1:5185**. Quem fala com o mundo é o nginx.
- `client_max_body_size 25m` — o painel envia foto em base64 dentro do JSON; o padrão de 1 MB devolveria 413.
- `proxy_set_header X-Real-IP` e `X-Forwarded-Proto` são obrigatórios: sem eles a aplicação vê todo mundo como 127.0.0.1, o contador registra um visitante só e o cookie não recebe a marca Secure.

- `error_page 502 503 504` serve a página de manutenção quando a aplicação está fora do ar.

PONTO DE ATENÇÃO - restrição de systemd

Não usar `MemoryDenyWriteExecute` na unit do `systemd`. Essa trava mata o motor JavaScript com 5/TRAP segundos depois de subir, e o sintoma parece queda de rede. É uma armadilha já paga na infraestrutura deste conjunto de projetos.

9. Dependências e integrações externas

Serviço	Onde	Como é usado	Se cair
ViaCEP	server.js	Fonte primária da consulta de CEP	Cai para a BrasilAPI
BrasilAPI	server.js	Reserva da consulta de CEP	Responde 503 com “preencha o endereço à mão”
Google Fonts	site, /admin, /restrito	Cormorant Garamond, Figtree e Questrial	Fonte de sistema assume
Google Maps	site público	Mapa incorporado na seção de contato	Bloco vazio
Doctoralia	site público	Link externo de agendamento	Link quebrado
Unsplash	site público	Fotos do topo, atendimento online e capas do blog	Imagens não carregam
WhatsApp (wa.me)	site público	Botão flutuante e envio do formulário	Link quebrado
pg_dump	servidor	Backup do PostgreSQL	Backup da gestão falha; o do site continua. Mensagem clara aponta o pacote postgresql-client

Por que a consulta de CEP passa pelo servidor em vez de o navegador chamar direto: a CSP do /admin e do /restrito é `connect-src 'self'`, então a chamada externa seria bloqueada e afrouxar a política a enfraqueceria; além disso, o IP de quem digita o CEP não vai para um terceiro (LGPD), e dá para cachear e limitar num lugar só. O cache guarda 30 dias, com teto de 5.000 CEPs, limite de 60 consultas por minuto por IP e tempo máximo de 4 segundos por chamada.

NÃO IDENTIFICADO

IDs de GA4, GTM, Pixel, Clarity e Hotjar estão **vazios** em `assets/js/config.js`. A estrutura de carregamento condicionada ao aceite do cookie existe, mas nenhuma ferramenta de medição está ativa hoje.

10. Tratamento de erros

Situação	Comportamento
Exceção não prevista	Detalhe vai só para o log ; o cliente recebe <code>500 {"error": "Erro interno"}</code> . Mensagem de exceção costuma revelar caminho de arquivo e estrutura interna
Erro de SQL	O adaptador anexa o SQL à mensagem antes de relançar — vai apenas para o log do servidor, nunca para a tela. “relation X does not exist” sozinho não diz de onde veio
PostgreSQL fora do ar	Tentativa de religar; se falhar, 503 com página explicativa (HTML) ou JSON, mais Retry-After. Somente o /restrito é afetado
Cliente ocioso do pool com erro	Capturado por <code>pool.on("error")</code> . Sem esse ouvinte o Node derruba o processo inteiro — a clínica saíria do ar por uma conexão parada
Falha ao contar visita	Engolida em silêncio, por decisão explícita: medir acesso nunca pode derrubar a entrega da página
Campo numérico em branco	String vazia vira NULL ou volta ao valor padrão. Campo obrigatório em branco responde dizendo qual falta , em vez de 500
Rota inexistente	404 com <code>{"error": "Rota não encontrada"}</code> . No /restrito, a interface traduz esse 404 específico para “o serviço não foi reiniciado”, porque a tela é lida do disco a cada acesso mas as rotas ficam em memória
Chave de cifra ausente	O serviço se recusa a subir . Subir sem a chave significaria voltar a gravar prontuário em texto puro com a clínica trabalhando e ninguém percebendo
Migration com erro	Para o boot , em vez de sumir. É o oposto do modelo anterior, em que ALTER TABLE dentro de try/catch vazia engolia erro de digitação por anos

11. Testes

Duas suítes versionadas no repositório, ambas executadas pelo pipeline antes de qualquer entrega. Não há framework de teste: são scripts Node com contagem própria de aprovações e falhas.

Suíte	Como roda	O que prova
testar-limitador.js	node testar-limitador.js	Comportamento do freio de tentativas: balde por IP, balde por conta, janela de bloqueio e persistência em arquivo
testar-campo-vazio.cjs	node testar-campo-vazio.cjs (porta 5197, exige PostgreSQL e DADOS_CHAVE)	Sobe o servidor de verdade , autentica e conversa por HTTP. Prova o caminho completo do formulário até o banco para as 32 colunas numéricas de 8 módulos

FATO

A suíte do campo vazio roda contra o **mesmo banco do cliente** quando executada no servidor. Por isso todo registro que ela cria leva o prefixo ZZ QA e é apagado **por id** no fim — nunca por LIKE, nunca por nome. É uma proteção deliberada contra apagar dado real.

Histórico de verificação registrado no projeto (suítes de apoio que rodaram durante o desenvolvimento e não estão versionadas): pentest das três superfícies 69/69, matriz de perfis de acesso 28/28, numeração 12/12, fluxo da regra de negócio 47/47 e 51/51, edição de lançamento 19/19, paciente ativo/inativo 27/27, relação de pacientes 22/22, horários 20/20.

PONTO DE ATENÇÃO - cobertura não reproduzível

As suítes acima ficavam em uma pasta de rascunho e **não estão no repositório**. Os resultados são registro histórico, não verificação reproduzível hoje: quem assumir o projeto não consegue reexecutá-las. Somente as duas suítes da tabela acima rodam no pipeline.

11.1 O que não é coberto por teste automatizado

- A **publicação** do site (geração das páginas estáticas a partir de src/).
- O painel /admin: nenhuma suíte exercita suas rotas.
- A **cifra e a decifra** ponta a ponta — há conferência dentro de cifrar-dados.js, mas ela é um passo de migração, não um teste do pipeline.
- As **impressões**, que são a entrega mais visível do sistema de gestão.
- A restauração de backup em servidor real.

12. Pontos de atenção

Itens observados na análise. Cada um está classificado como **fato verificado no código**, **inferência** ou **recomendação**.

12.1 Documentação interna desatualizada

O README.md do projeto descrevia uma arquitetura que **não é mais a atual**. Ele foi escrito antes da migração para PostgreSQL e não havia sido revisto.

O que o README.md afirmava	O que o código mostra
"Stack: Node puro + SQLite via better-sqlite3"	O /restrito usa PostgreSQL desde a v1.19.0; o SQLite ficou só no site e no painel
Tabela das áreas: /restrito usa data/gestao.db	Usa a base bemestar_gestao. O arquivo gestao.db em data/ é o legado da virada , e foi explicitamente removido da lista de backup
"Banco de dados: SQLite, dois arquivos"	Dois motores diferentes, um para cada área
Backup: VACUUM INTO para os dois bancos	Só o site. A gestão usa pg_dump
"as anamneses são visíveis a todos os profissionais; os lançamentos, não"	Invertido pela migration 003 e pela v1.26.0 : o profissional vê apenas os prontuários e as anamneses <i>delete</i>
Anamneses tem submenu (Todas / Psicanálise / Ozonioterapia / Integrativas)	O submenu foi removido na v1.14.0; hoje é um item único e o modelo se escolhe dentro do formulário
Não mencionava criptografia, auditoria, arquivamento nem migrations	Todos existem (v1.21.0, v1.22.0, v1.28.0 e v1.19.0)

RECOMENDAÇÃO

O README foi reescrito como parte desta análise. A recomendação que permanece é de processo: **a atualização do README deveria fazer parte da subida de versão**, junto da entrada no changelog — que, essa sim, está rigorosamente em dia nas duas séries (CHANGELOG.md para o site, HISTORICO_VERSOES para a gestão).

FATO

Vale registrar o contraste: o projeto mantém **dois changelogs impecáveis**, com o raciocínio de cada mudança, e comentários de código que explicam decisões em profundidade incomum. A documentação que envelheceu foi justamente a de entrada — o README —, que é a primeira que alguém novo lê. O problema aqui não é falta de cultura de documentação; é o ponto do processo em que ela não foi acionada.

12.2 Comentários do código desatualizados

O cabeçalho de restrito.js (linhas 6 e 17) ainda descreve "banco próprio: data/gestao.db" e instrui a proteger o gestao.db no deploy. O arquivo migrou para PostgreSQL nove versões atrás. É um comentário, não código — não quebra nada, mas induz ao erro quem abrir o arquivo pela primeira vez.

12.3 Contradição interna no POSTGRES.md

O mesmo arquivo afirma, na seção de operação, que "os dumps saem **cifrados**" e que "um backup que vaze não entrega prontuário nenhum"; e afirma, na seção de pendências, que falta destino externo com criptografia porque "as cópias são a clínica inteira **em texto puro**".

INFERÊNCIA

Pelo código, a primeira afirmação é a correta para os campos listados em CAMPOS_PROTEGIDOS. A segunda parece ser um parágrafo sobrevivente da época anterior à criptografia. Vale notar que ela não é inteiramente falsa: **nome e código do paciente continuam em texto legível no dump**, por decisão de projeto. Um dump vazado revela quem são os pacientes da clínica, ainda que não revele CPF, endereço nem prontuário.

12.4 Quatro números de versão diferentes

Onde	Valor	O que versiona
APP_VERSION (server.js)	1.20.0	Site e painel /admin
SISTEMA_VERSION (restrito.js)	1.28.0	Sistema de gestão
package.json	1.15.0	O pacote
package-lock.json	1.11.0	Cópia do anterior, desatualizada

INFERÊNCIA

As duas primeiras são **intencionais** e estão documentadas: o site e a gestão têm séries próprias, e o cliente acompanha a da gestão. As duas últimas parecem acompanhamento incompleto — o package.json acompanha a série do site com um passo de atraso, e o lock ficou quatro versões atrás do próprio package.json. Desde 12/08/2026 a série do site tem changelog próprio em CHANGELOG.md, e a da gestão continua em HISTORICO_VERSIONES, dentro do restrito.js.

RECOMENDAÇÃO

Definir explicitamente o que o package.json versiona (sugestão: espelhar o SISTEMA_VERSION, que é o número que o cliente vê) e regenerar o lock na subida. Alternativamente, congelar o package.json em 0.0.0 e assumir que as versões reais são as duas constantes — o pior cenário é o atual, em que os quatro números existem e nenhum concorda.

12.5 Integridade referencial ausente no banco

O esquema do PostgreSQL **não declara nenhuma FOREIGN KEY**. Todos os relacionamentos (paciente, profissional, sala, convênio, procedimento, prontuário) existem apenas como colunas *_id mantidas pela aplicação. O detalhamento está no documento de Banco de Dados.

FATO

Esse desenho já produziu um defeito real em produção, corrigido na v1.18.0: três anamneses apontavam para uma pasta de prontuário que não existia mais. A lista não achava a pasta e mostrava “sem prontuário”; o formulário via o campo preenchido e escondia o botão Excluir. O registro ficou impossível de apagar, sem motivo visível. Existe hoje uma migração no boot que solta vínculos órfãos — é um remendo no lugar da restrição.

12.6 Desempenho da busca de pacientes

Descrito no item 4.5: a listagem de pacientes traz **todas** as linhas do banco para filtrar em memória, porque o CPF cifrado não é comparável em SQL. A paginação acontece na tela.

INFERÊNCIA

Para a escala atual (dois a três profissionais, cerca de 9 mil atendimentos por ano) isso é irrelevante. O ponto de atenção é que o custo cresce linearmente e não aparece como erro — só como lentidão. A alternativa já construída é a impressão digital HMAC, que resolve igualdade exata mas não busca parcial.

12.7 Backup sem destino externo

As cópias ficam em backups/, na **mesma máquina**. Isso cobre erro humano e corrupção, não perda do servidor. A pendência está reconhecida no README e no POSTGRES.md.

RECOMENDAÇÃO

Definir destino externo com criptografia em trânsito e em repouso. E **ensaiar a restauração**: o mecanismo está testado, o procedimento no servidor real nunca foi executado. Restauração não ensaiada é backup não comprovado.

12.8 Chave de cifra é ponto único de perda

PONTO DE ATENÇÃO

Perder a DADOS_CHAVE torna ilegíveis CPF, RG, endereço, telefone, e-mail, anamneses, prontuários e auditoria — **inclusive nos backups**. O próprio POSTGRES.md alerta para guardar cópia fora do servidor. Não há, no repositório, procedimento documentado de custódia dessa chave: onde fica a cópia, quem tem acesso, como se recupera.

12.9 Histórico do repositório

O arquivo data/site.db permanece no histórico de dois commits antigos, contendo hash de senha e o sal do contador de visitas. Limpar exige reescrever o histórico do repositório.

12.10 Síntese

Item	Natureza	Gravidade	Esforço
12.1 README desatualizado	Documentação	Média	Resolvido nesta entrega
12.2 Comentários desatualizados	Documentação	Baixa	Baixo
12.3 Contradição no POSTGRES.md	Documentação	Média	Baixo
12.4 Quatro versões divergentes	Processo	Baixa	Baixo
12.5 Sem FOREIGN KEY	Dívida técnica	Alta	Médio
12.6 Busca carrega tudo em memória	Desempenho	Baixa hoje	Médio
12.7 Backup sem destino externo	Risco operacional	Alta	Médio
12.8 Custódia da chave de cifra	Risco operacional	Alta	Baixo
12.9 Banco no histórico do git	Segurança	Média	Alto
Suítes de apoio não versionadas	Qualidade	Média	Médio

FATO

Nota de método. Este documento descreve o sistema como ele está, e não como deveria estar. Onde o código e a documentação existente divergem, prevaleceu o código, e a divergência foi registrada. Nenhum endpoint, tabela, variável ou funcionalidade citado aqui foi inferido: todos foram lidos nos arquivos do projeto. Nenhuma senha, chave, token ou credencial real foi reproduzida.

PONTO DE ATENÇÃO - alvo em movimento

Estado capturado. Durante esta análise o projeto **recebeu uma entrega**: o site passou de 1.19.0 para **1.20.0** (rodada de SEO e compartilhamento), o package.json foi para 1.15.0 e o CHANGELOG.md foi criado. O documento foi revisto para refletir esse estado. O sistema de gestão (1.28.0) e o esquema do banco não foram alterados nessa entrega. **Qualquer versão posterior a 1.20.0 do site ou a 1.28.0 da gestão não está coberta aqui.**